

SALTY: Explainable Artificial Intelligence Guided Structural Analysis for Hardware Trojan Detection

Tanzim Mahfuz¹, Pravin Gaikwad², Tasneem Suha¹, Swarup Bhunia², and Prabuddha Chakraborty¹

¹Department of Electrical & Computer Engineering, University of Maine, Orono, ME, USA

²Department of Electrical & Computer Engineering, University of Florida, Gainesville, FL, USA

Abstract—Hardware Trojans are malicious modifications in digital designs that can be inserted by untrusted supply chain entities. Hardware Trojans can give rise to diverse attack vectors such as information leakage (e.g. MOLES Trojan) and denial-of-service (rarely triggered bit flip). Such an attack in critical systems (e.g. healthcare and aviation) can endanger human lives and lead to catastrophic financial loss. Several techniques have been developed to detect such malicious modifications in digital designs, particularly for designs sourced from third-party intellectual property (IP) vendors. However, most techniques have scalability concerns (due to unsound assumptions during evaluation) and lead to large number of false positive detections (false alerts). Our framework (SALTY) mitigates these concerns through the use of a novel Graph Neural Network architecture (using Jumping-Knowledge mechanism) for generating initial predictions and an Explainable Artificial Intelligence (XAI) approach for fine tuning the outcomes (post-processing). Experiments show $> 98\%$ True Positive Rate (TPR) and True Negative Rate (TNR), significantly outperforming state-of-the-art techniques across a large set of standard benchmarks.

Index Terms—Hardware Trojans, Hardware Security and Trust, Explainable Artificial Intelligence

I. INTRODUCTION

The global distributed supply chain has a massive economic advantage but suffers from security and trust concerns. Most modern semiconductor companies operate in a fabless model, including economic giants such as NVIDIA, Apple, AMD, Qualcomm, and Broadcom that are driving the current AI wave. Most small semiconductor business are following the same example set by these larger companies. Most of these companies often acquire/license application-specific design blocks (also called IPs) from third-party vendors around the world, integrate these IPs into their core design, and send the final design to other entities for fabrication/testing/packaging. This distributed model opens up the system to a variety of threat vectors such as hardware Trojans, intellectual property theft, and privacy compromise through reverse engineering.

In this study, we focus on hardware Trojans which are intentional malicious modifications in digital designs that can lead to certain non-characteristic behavior of the system in-field. Such attacks can be carried out by malicious third-party IP vendors (3PIP), rogue employees in the design house, and an untrusted foundry. Typically hardware Trojans have a very small footprint (compared to the original design) in terms of area/power/delay and are rarely triggered during the

system testing phase. Hence, it is a difficult task to detect these modifications in massive digital designs. Recent works such as VIPR [1], [2] and TrojanSAINT [3] have attempted to detect these Trojans in digital designs using different Artificial Intelligence (AI) techniques with varying degrees of success. VIPR suffers from low True Positive Rate (TPR) while TrojanSAINT has massive instability issues (performs poorly on certain designs). The post-processing techniques introduced in VIPR are inspiring, but lack dynamism and flexibility. Other techniques such as GNN4Gate [4], FASTGO [5], NHTD [6] have major scalability/practicality concerns because of unsound evaluation choices (trained and tested on similar benchmark structures).

In this work, we propose SALTY (Structural AI for Explainable Trojan Analysis), a novel hardware Trojan detection framework (golden free) that attempts to mitigate the concerns and shortcomings of existing frameworks. SALTY utilizes a novel Graph Neural Network (GNN) with Jumping Knowledge (JK) mechanism while leveraging design structural features similar to SAIL [7] (a reverse engineering framework). The JK-mechanism reduces the risk of over-smoothing by aggregating hidden layer embeddings. This allows the model to attain a much higher TPR and TNR compared to other state-of-the-art GNNs. Artificial Intelligence (AI) and statistical models are often not fully aware of the domain constraints and make irrational decisions (hallucination) leading to poor performance. To tackle this concern, we utilize explainability techniques to draw insights about when and how our AI models fail to make correct decisions. This insight allows us to build a dynamic post-processing module that can further enhance the performance of SALTY. We evaluate our framework on a large set of standard TrustHub benchmarks (> 15) and compare the results against seven other state-of-the-art (SOTA) Trojan detection techniques. We observe significant performance improvements (TPR/TNR combined) and enhanced visibility into the Trojan detection process (via explainability). We make the following contributions:

- 1) Designed a novel Graph Neural Network (GNN) that utilizes Jumping Knowledge (JK) mechanism for efficient hidden layer embedding aggregation.
- 2) Developed a novel explainability guided dynamic post-processing module that can boost Trojan detection performance by removing hallucinations.

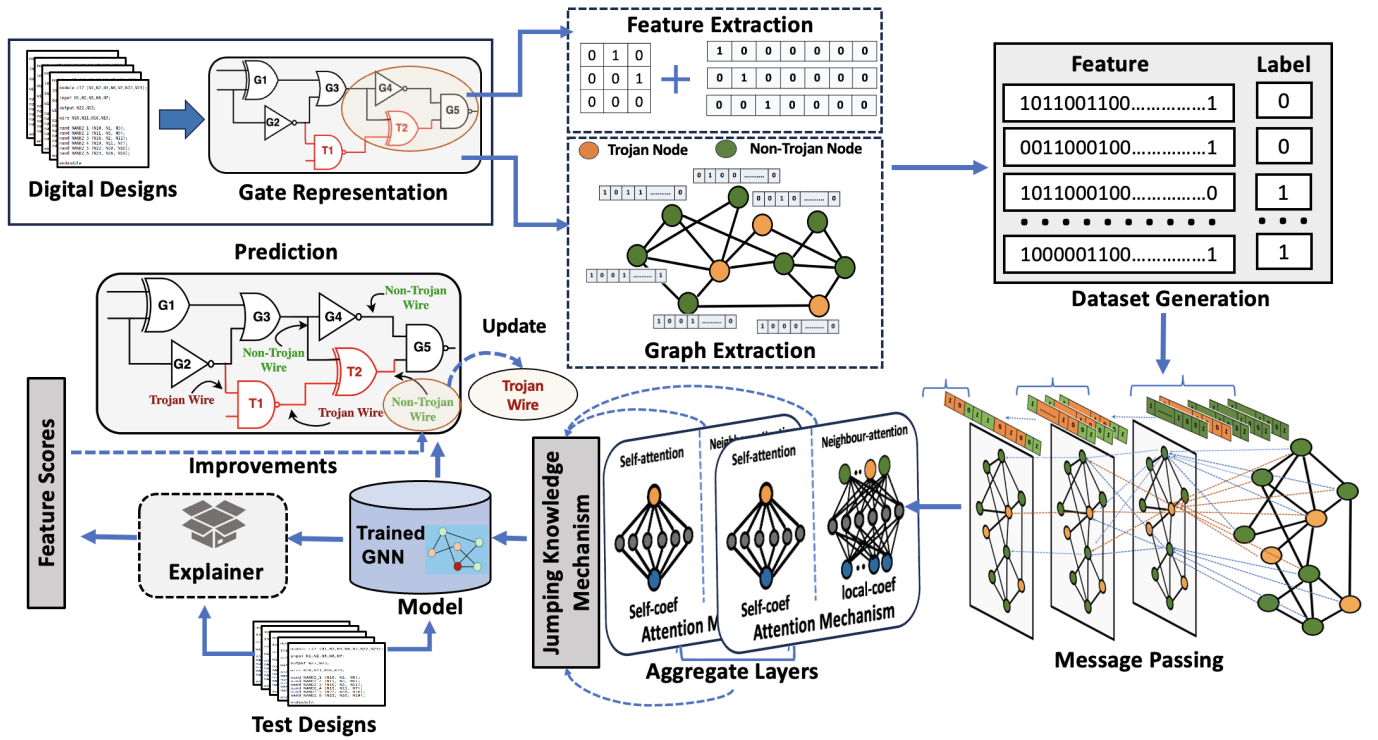


Fig. 1: The SALTY Framework: Input/output, feature extraction, graph neural network, and post-processing.

- 3) Implemented the SALTY framework as a highly parameterized tool and integrated it into the EDA-flow.
- 4) Robust generalization in hardware Trojan detection, enabling the identification of Trojans in entirely unseen benchmarks without prior exposure to similar structures.
- 5) Extensively evaluated SALTY with > 15 benchmarks against seven SOTA Trojan detection frameworks.

II. RELATED WORKS

A hardware Trojan (HT) is a malicious alteration or addition to a hardware design during its creation or assembly. These Trojans compromise the hardware by creating vulnerabilities or unintended functions, ranging from small circuits leaking data to system-disrupting modifications. Their stealth and dormancy make detection and mitigation difficult. Detecting hardware Trojans (HTs) remains a crucial research focus, prompting the development of various frameworks and methods to tackle this issue. In this section, we examine significant progress made in the domain, particularly emphasizing machine learning and graph-based approaches, and discuss their advantages and drawbacks.

VIPR improves detection accuracy without relying on golden models through pseudo-self-referencing and post-processing, yet it is limited by its dependence on synthetic Trojans and feature selection. A hybrid XGBoost framework [8] integrates static and dynamic analysis but faces issues with scalability and generalizability due to computational demands and dataset reliance. Graph Neural Networks (GNNs) are effective in representing circuits as graphs, facilitating accurate detection. TrojanSAINT breaks circuits into subgraphs to extract features, although it experiences challenges with

reduced accuracy and scalability. GNN4Gate facilitates automatic Trojan detection without the need for golden models; however, it necessitates the manual design of various features, which complicates its application to real-world scenarios. GNN4HT [9] expands detection to gate-level and RTL designs, but is hampered by high interclass similarity and limited generalization due to insufficient data set augmentation. FAST-GO employs Graph Convolutional Networks (GCNs) enhanced with improved features and dynamic thresholds for scalable detection. R-HTDetector [10] boosts robustness through adversarial training. NHTD-GL automates feature extraction to detect threats at the node level. These methods operate at the net level and encounter difficulties with imbalanced datasets and feature initialization, while also lacking the ability to localize Trojans.

III. MOTIVATIONS

A. Generalization to Entirely Unseen Digital Designs

A key motivation for our work is to achieve robust generalization in hardware Trojan detection across completely unseen design. Therefore, a detection framework must be capable of identifying Trojans in completely new designs without having been trained or validated on similar structures. Table I highlights the summary of current approaches.

B. Informed Post Processing - Minimize AI Hallucinations

While techniques like VIPR have investigated post-processing, such approaches are static in nature and fail to take the advantage of more intricate patterns. We hypothesize that explainability techniques [11]–[15] can provide deeper

TABLE I: Summary of the state-of-the-art Trojan detection frameworks and comparison with SALTY.

Research Study	ML Technique	Data Balancing Technique	Post Processing	Model Training Applicability	No Validation in SCF [†]	No Training in SCF [†]	Model Insight
VIPR* [1]	SVM	Min Max Scaler	Yes	Practical	N/A	N/A	✗
TrojanSAINT [3]	GCN	Threshold Tuning	No	Practical	✗	✓	✗
XGB [8]	XGB	Class Weights	No	Not Practical	N/A	✗	✗
GNN4Gate [4]	GCN	Class Weights	No	Not Practical	✗	✗	✗
GNN4HT [9]	GIN	Data Augmentaion	No	Not Practical	✗	✗	✗
FAST-GO [5]	GCN	Class Weights	No	Not Practical	✗	✗	✗
R-HTDET [10]	GAN	Oversampling	No	Not Practical	✗	✗	✗
NHTD [6]	GIN + GAT	Custom Sampling	No	Not Practical	✗	✗	✗
SALTY	GAT + JK	Class Weights	Yes	More Practical	✓	✓	✓

† Refers to Same Circuit Family. For example, rs232_t1000, rs232_t1100 belong to same circuit family. VIPR* uses pseudo-self referencing for training.

Algorithm 1: Model Generator

Input: $\mathcal{D}, \mathcal{T}_{lb}, \mathcal{L}$
Output: $\mathbb{M}$

```

1  $G_r \leftarrow \text{graphify}(\mathcal{D})$ 
2  $E_d \leftarrow \text{edging}(G_r)$ 
3  $X\_data, Y\_data \leftarrow \emptyset; \mathcal{D}_{wires} \leftarrow G_r.wires$ 
4 for  $w$  in  $\mathcal{D}_{wires}$  do
5    $S_f \leftarrow \text{structural\_features}(G_r, \mathcal{L}, w)$ 
6    $X\_data.append(S_f)$ 
7    $Y\_data.append(\mathcal{T}_{lb}[w])$ 
8  $M_d \leftarrow \text{build\_model}(GAT, JK)$ 
9  $\mathbb{M} \leftarrow \text{training}(M_d, X\_data, Y\_data, E_d)$ 
10 return  $\mathbb{M}$ 

```

insight into AI models opening pathways for more robust post-processing that can foster trust and minimize the AI hallucinations.

IV. METHODOLOGY

Inspired by Sec. III, we have developed the SALTY framework which: (i) Integrates a Jumping Knowledge mechanism to boost performance on unseen designs; (ii) Utilize explainable AI for reducing AI Hallucinations. The overview of the SALTY is illustrated in Fig.1.

A. Knowledge Discovery & Model Design

Let's assume, a graph $G_r = (V, E)$ is defined, where V represents vertices and E denotes hyperedges. Here, V : wires in a digital design, and E : their connections. Algo. 1 outlines the initial stages from graph generation to model creation using the Trojan inserted design ($\mathcal{D}$), ground truth ($\mathcal{T}_{lb}$) for wire labels, and locality ($\mathcal{L}$) as inputs. Locality ($\mathcal{L}$) takes care of the number of neighbors that needs to be collected from the subgraph. In line 1, the *graphify* constructs G_r , with connection details E_d (edge-index), in line 2. Subsequently, all nodes ($\mathcal{D}_{wires}$) of the graph are gathered, and the structural features of each wire and the corresponding true label from ($\mathcal{T}_{lb}$) are extracted and organized into the dataset (X_data, Y_data)(lines 4-7). SALTY employs local structural features of the sub-design, similar to the approach used in SAIL [16] and SAIL-based works [17]. The structural features of a wire describe its local placement and how it connects to other wires

through the gates it interacts with. In a subgraph, we encode the connectivity using an adjacency matrix and represent gate types through one-hot encoding, similar to [11]. Fig. 1 illustrates the structural features extraction process, showing how the gates are vectorized and how the dataset is created via feature extraction and labeling. We utilize the Breadth-First Search (BFS) algorithm to explore neighboring gates around a specific wire - the one for which we are extracting features and assigning a label. The dataset is used for training and the trained model, $\mathbb{M}$ is stored in line 10. In this work, We employ Graph Attention Networks (GATs) [18] with a Jumping Knowledge (JK) mechanism [19], which incorporates attention to prioritize key graph regions, essential for detecting subtle Trojan patterns. Given a graph with node features f_i for each node $i \in V$, a single GAT layer computes the updated node features f'_i as follows:

$$f'_i = \sigma \left(\sum_{j \in X(i)} \alpha_{ij} W f_j \right)$$

where: $X(i)$, set of neighboring nodes; W , weight matrix; σ , non-linear activation function; α_{ij} , attention coefficients computed by:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(v^\top [W f_i \parallel W f_j]))}{\sum_{k \in X(i)} \exp(\text{LeakyReLU}(v^\top [W f_i \parallel W f_k]))}$$

Here, v is a learnable weight vector. The JK mechanism combines information from different layers of GATs, addressing the issue of over-smoothing. If we have node representations from L layers, $\{f_i^{(1)}, f_i^{(2)}, \dots, f_i^{(L)}\}$, the JK aggregates into a final node embedding f_i^{JK} using:

$$f_i^{JK} = \parallel_{l=1}^L f_i^{(l)}$$

This approach retains information from all layers without loss. SALTY consists of two sequential GAT layers aggregating with Jumping Knowledge which are fed into *softmax* output layer. For classification, we assign each node to the class corresponding to the highest predicted probability.

B. Explainable AI: Post Processing Development

To introduce interpretability, we have used PyTorch Geometric's Explainer module, designed to identify influential nodes and edges in GNNs [20]. Within this module, we

Algorithm 2: XAI Development

Input: $\mathcal{D}, \mathbb{M}, \mathcal{T}_{lb}, \mathcal{L}, n, \mathcal{T}_h, \mathcal{N}$
Output: TPR, TNR

```

1  $G_r \leftarrow \text{graphify}(\mathcal{D})$ 
2  $E_d \leftarrow \text{edging}(G_r)$ 
3  $S_f \leftarrow \text{structural\_features}(G_r, \mathcal{L}, G_r.\text{wires})$ 
4  $P_0, P_1 \leftarrow \text{prediction}(\mathbb{M}, S_f, E_d, G_r.\text{wires})$ 
5  $x, y \leftarrow \text{len}(P_0), \text{len}(P_1)$ 
6  $F_0[x][n], F_1[y][n] \leftarrow \text{explainer}(\mathbb{M}, P_0, P_1, n)$ 
7 for  $j$  in  $\text{range}(n)$  do
8    $\mathcal{V}[0][j] \leftarrow \text{sum\_avg}(F_0[i][j] : i \leftarrow \text{range}(\text{len}(F_0)))$ 
9    $\mathcal{K}[0][j] \leftarrow \text{sum\_avg}(F_1[i][j] : i \leftarrow \text{range}(\text{len}(F_1)))$ 
10 for  $i$  in  $\text{range}(F_0)$  do
11    $\mathcal{C}_1 \leftarrow \text{sum}(F_0[i][n]) \geq (\text{sum}(\mathcal{V}[0][n]) * \mathcal{N})$ 
12    $\mathcal{C}_2 \leftarrow |\text{sum}(F_0[i][n]) - \text{sum}(\mathcal{K}[0][n])| \geq \mathcal{T}_h$ 
13   if  $\mathcal{C}_1$  or  $\mathcal{C}_2$  then
14      $\text{switch\_pred}(P_0, P_1)$ 
15 TPR, TNR  $\leftarrow \text{compute\_metrics}(P_0, P_1, \mathcal{T}_{lb})$ 
16 return TPR, TNR

```

have leveraged the *Captum-Explainer* algorithm [21]. While other frameworks like SHAP [22] and LIME [12] are widely used for model interpretability, they may not effectively capture the complex relationships inherent in graph data. We utilize *Integrated Gradients* [23], a gradient-based attribution method from *Captum-Explainer* that quantifies the contribution of each input feature to the model’s prediction by integrating the gradients along a path from a baseline input to the actual input. For an input $\mathbf{x}$ and a baseline $\mathbf{x}'$, the attribution $IG_{f_i}(\mathbf{x})$ for feature f_i is calculated as:

$$IG_{f_i}(\mathbf{x}) = (x_{f_i} - x'_{f_i}) \times \int_{\alpha=0}^1 \frac{\partial F(x' + \alpha \times (x - x'))}{\partial x_{f_i}} d\alpha$$

where F is the model’s output and α is a scaling factor between 0 and 1. Algo. 2 explains the automated XAI process that enhances framework performance. Lines 1–3 extract the graph, edge-index, and structural features from the test design. Line 4 uses the trained model to predict non-Trojan and Trojan wires and lengths stored in x and y . The parameter n specifies the number of top feature scores that the *Captum-Explainer* module will provide. We eventually obtain top n feature scores for non-Trojan and Trojan nets from F_0 and F_1 , respectively, which effectively distinguish between the two types of wires, in line 6. Aiming to detect Trojan nodes as accurately as possible, we take the top n average feature scores for both Trojan and non-Trojan wires (denoted as $\mathcal{K}$ and $\mathcal{V}$ array), in lines 7–9. The inputs $\mathcal{N}$ and $\mathcal{T}_h$ are crucial, as they determine when a non-Trojan node, initially predicted as non-Trojan (a false negative) should be reclassified as Trojan (a true positive). Specifically, $\mathcal{N}$ acts as a multiplier to check if summation of feature scores of a non-Trojan net is equal to or exceeds $\mathcal{N}$ times summation of the average feature scores of non-Trojan nets, line 11. Similarly, $\mathcal{T}_h$ represents the minimum allowable difference between summation of feature scores of a non-Trojan net and summation of the average feature

TABLE II: SALTY for different localities.

Benchmarks	Locality = 5		Locality = 7		Locality = 10	
	TPR	TNR	TPR	TNR	TPR	TNR
rs232_t1000	40.00	100	100	96.63	100	96.03
rs232_t1100	36.36	79	100	94.88	100	93.67
rs232_t1200	100	72.04	100	96.31	100	94.41
rs232_t1300	57.14	92.13	100	96.32	100	98.36
rs232_t1400	100	72.70	100	95.93	100	92.46
rs232_t1500	63.64	78.74	100	94.61	100	92.69
rs232_t1600	42.86	93.07	100	97.66	100	99.67
rs232_t1700	70	92.74	100	98.33	100	99.67
s15850_t100	100	98.76	100	99.70	100	99.43
s38417_t100	91.67	99.78	100	100	91.67	100
s38417_t200	66.67	99.93	93.33	93.33	73.33	100
s38417_t300	100	99.93	100	100	94.12	100
s38584_t100	38.89	99.89	88.89	99.98	55.56	100
s35932_t100	86.67	98.68	93.33	100	86.67	100
s35932_t200	66.67	98.71	100	99.91	75.00	100
s35932_t300	66.67	98.86	100	99.96	75	99.97
Average	70.45	92.18	98.47	98.14	90.71	97.88

scores of Trojan nets required for reclassification, line 12. If either condition is met, we adjust the prediction accordingly (prediction switched) in line 14 and report our results in form of TPR, TNR.

V. RESULTS & ANALYSIS

A. Experimental Setup

To validate our proposed framework, we have established a comprehensive experimental setup to assess its effectiveness. In both algorithms, we set the parameter $\mathcal{L} = 7$. For algorithm 2 specifically, we have used $\mathcal{N} = 2$, $\mathcal{T}_h = 0.1$, and $n = 10$ to inform decisions based on the XAI features. Our experiments utilize digital designs obtained from Trust-Hub [24]. Regarding training parameters, we employed a weighted binary cross-entropy loss function with a batch size of 64, the Adam optimizer, and learning rate 0.005. 20% of the training data was set aside for validation purposes. Thus, training, validation, and test datasets are mutually exclusive and unknown to each other. The number of epochs is determined using early stopping criteria based on the validation set. We assess the performance of the proposed method using the metrics of True Positive Rate (TPR) and True Negative Rate (TNR). The TPR, also known as sensitivity, quantifies the proportion of actual positive cases (Trojan nodes) that are correctly identified by the model. Conversely, the TNR, or specificity, measures the proportion of actual negative cases (non-Trojan nodes) that are accurately classified. These metrics are calculated using the following equations:

$$TPR = \frac{TP}{TP + FN} \quad TNR = \frac{TN}{TN + FP}$$

B. SALTY for Different Locality Sizes

We also experiment the performance of SALTY with different locality sizes ($\mathcal{L}$) for structural feature extraction, specifically sizes of 5, 7, and 10. The results, summarized in Table II, indicate that $\mathcal{L} = 7$ yields the highest accuracy in detecting hardware Trojans. Our observation finds that $\mathcal{L} = 7$ can be attributed to an optimal balance between context and relevance. A smaller locality size, such as 5, may not capture sufficient structural information surrounding each node, potentially missing critical patterns associated with

TABLE III: Quantitative comparison between SALTY and other state-of-the-art frameworks. SALTY outperforms all.

Benchmarks	VIPR [1]		TrojanSAINT [3]		GNN4Gate [4]		GNN4HT [9]		FAST-GO [5]		R-HTDET [10]		NHTD [6]		SALTY			
	TPR	TNR	TPR	TNR	TPR	TNR	TPR	TNR	TPR	TNR	TPR	TNR	TPR	TNR	GAT+JK TPR	GAT+JK TNR	XAI-GAT+JK TPR	XAI-GAT+JK TNR
rs232_t1000	100	98.68	100	60	100	100	100	99	100	84.36	100	94.30	100	100	100	96.69	100	96.63
rs232_t1100	50	98.71	92	68	100	100	100	99	100	84.30	100	93.30	100	99.66	100	95	100	94.88
rs232_t1200	76.47	99.67	41	80	100	100	100	99	100	84.77	93.20	96.50	100	99.66	100	96.38	100	96.31
rs232_t1300	100	99.67	100	74	100	100	100	99	100	83.13	100	94.40	100	100	100	96.39	100	96.32
rs232_t1400	92.31	98	92	50	100	100	100	99.50	100	84.36	100	97.80	100	99.66	100	96	100	95.93
rs232_t1500	91.67	99.67	71	82	100	100	100	99	100	84.77	98	94.30	100	100	100	94.68	100	94.61
rs232_t1600	100	99.67	73	57	100	100	100	100	100	84.02	97.40	92.50	69.23	100	100	97.69	100	97.66
rs232_t1700	100	100	-	-	-	-	-	-	-	-	-	-	-	-	100	98.35	100	98.33
s15850_t100	-	-	35	97	-	-	59.26	90.35	100	80.93	60.50	90.10	100	100	100	99.70	100	99.70
s38417_t100	100	99.91	92	92	100	99.92	100	95.62	91.67	80.15	77.30	99.90	66.67	100	100	99.99	100	99.99
s38417_t200	100	99.98	40	99	80	99.85	100	97.24	100	80.20	84	98.70	100	100	80	99.90	93.33	99.98
s38417_t300	-	-	98	96	100	99.46	53.33	95.13	100	80.21	83	99.90	93.33	99.98	94.12	100	100	100.00
s38584_t100	-	-	100	95	-	-	88.89	87.72	68.42	80.13	-	-	33.33	100	72.22	99.98	88.89	99.98
s35932_t100	-	-	100	100	93.33	99.98	93.33	96.19	100	80.19	80	75.20	80	100	80	100	93.33	100
s35932_t200	-	-	100	100	68.75	100	93.75	91.83	70.59	80.13	36.40	99.90	73.33	100	83.33	100	100	99.91
s35932_t300	-	-	97	100	33.33	100	100	97.97	100	80.45	88	100	80.77	99.95	83.33	100	100	99.96
Average	91.04	99.40	82.07	83.33	90.42	99.94	92.57	96.44	95.38	82.14	85.56	94.77	86.44	99.93	93.31	98.18	98.47	98.14

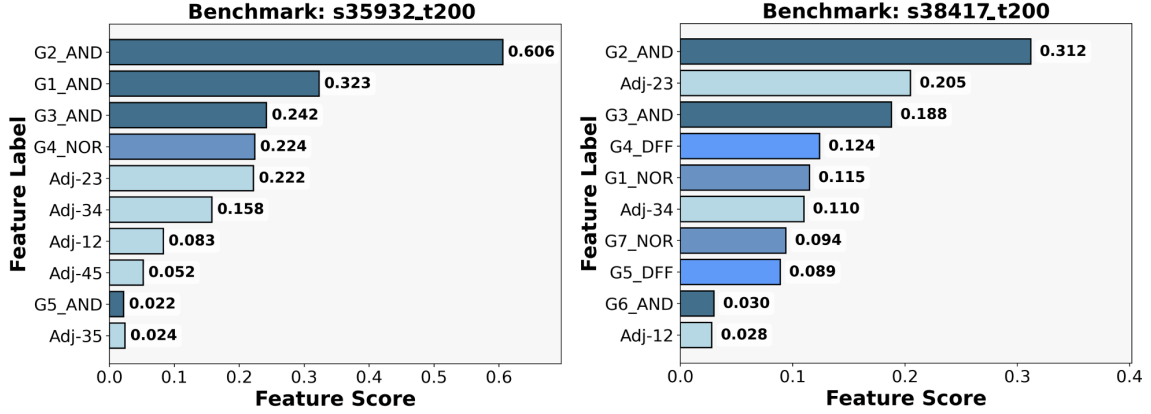


Fig. 2: Illustration of SALTY model's decision-making process for Trojan nodes. Two distinct benchmarks are shown here.

Trojan circuits. On the other hand, increasing the locality size to 10 or more introduces a larger context but also brings in additional irrelevant or noisy information.

C. Comparison with State-of-the-Art

We further evaluate our proposed framework against several state-of-the-art hardware Trojan detection methods, including VIPR, TrojanSAINT, GNN4Gate, GNN4HT, FAST-GO, R-HTDET, and NHTD. We evaluate the performance of several GNN architectures on GCN, GCN+JK, GAT, GAT+JK, and XAI-enhanced GAT+JK. As summarized in Table III, XAI-enhanced GAT+JK of SALTY outperforms these existing methods by achieving a TPR of 98.47% and a TNR of 98.14%. In our approach, we ensure that no benchmarks from the same circuit family are included in either the training or validation datasets. For instance, benchmarks like s38417_t100, s38417_t200, and s38417_t300, which belong to the same circuit family of s38417, are excluded from training and validation when evaluating any benchmark from s38417 (shown in Table I). This methodology closely mirrors practical applications where the test data is entirely unknown during the model development phase. Prior works such as GNN4Gate,

FAST-GO, R-HTDET and others include benchmarks from the same circuit family in their training sets potentially reduce generalization to new designs. Among them, TrojanSAINT employs a practical training approach, but exhibits a TPR of 82.07% and a TNR of 83.33%. As including designs from the same circuit family can potentially enhance model familiarity, NHTD is more effective at correctly identifying non-Trojan nodes, but less capable of detecting actual Trojan instances.

D. Important features for Trojan nodes

Figure 2 provides a comprehensive illustration of the feature scores conducted using XAI for detecting Trojan nodes. The analysis spans across two distinct benchmarks: s35932_t200 and s38417_t200. Each plot represents the top $n = 10$ features ranked by their contribution to the model's decision-making process. The aim is to showcase what features are important for the Trojan nodes. $G1$, $G2$ represent gates in the subgraph, with numerical labels denoting their order and values next to them indicate the logic gate type when structural features are gathered using BFS. $Adj-23$ indicates adjacency connections between $G2$ and $G3$. Observation from s38417_t200: The

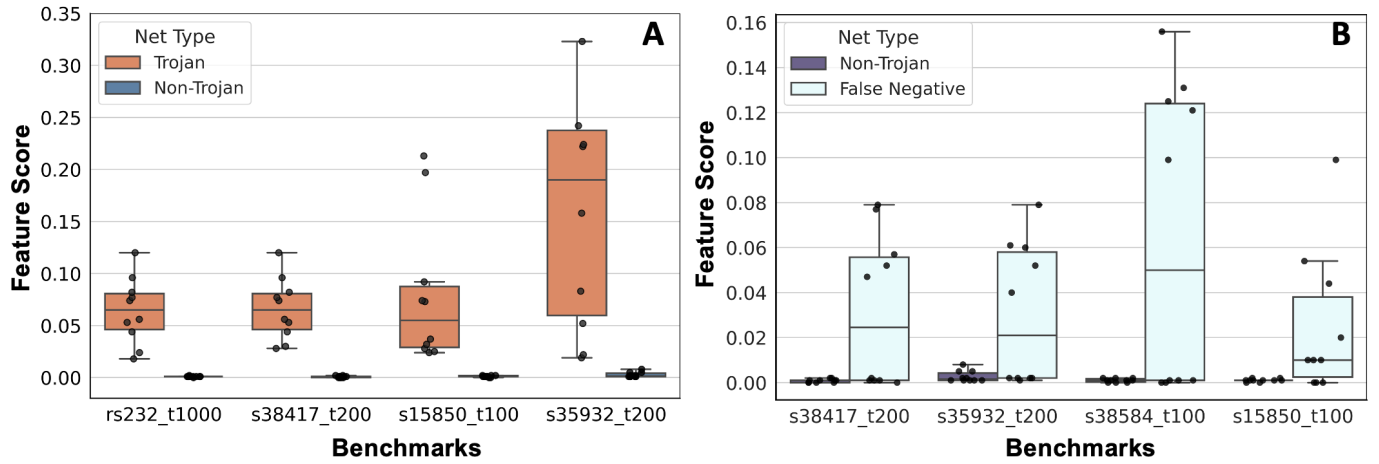


Fig. 3: **A**: Comparison of feature scores between Trojan and non-Trojan nodes - Trojan nodes exhibit significantly higher feature scores compared to non-Trojan nodes; **B**: Analysis of feature scores for non-Trojan nodes - Certain non-Trojan nodes generate high feature values, indicating false negatives in the classification.

TABLE IV: XAI generated human-interpretable rules for Trojan detection.

Benchmarks	What the AI Model learned to classify Trojan
s38417_t200	If [G2 = AND, G3 = AND && G4, G5 = D Flip Flop, G1 = NOR && G2, G3 are connected && G3, G4 are connected] is the locality of a net then there is a higher possibility that it can be a Trojan.
rs232-t1000	If [G2 = OR, G3 = OR && G2,G3 connected to each other && G4 = AND, G7 is NONE gate && G3,G4 are also connected] is the locality of a net then there is a higher possibility that it can be a Trojan.
s15850-t100	If [G1, G2, G3, G4 all are AND gate && G2 is connected with G3 && G1 is connected to G2 && G5 = D Flip Flop && G4, G5 are connected] is the locality of a net then there is a higher possibility that it can be a Trojan.
s38584_t100	If [G6, G7 are NONE gate && G1, G2, G3 all are AND gate && G1,G2, G3 are connected among it-selves] is the locality of a net then there is a higher possibility that it can be a Trojan.
s35932-t200	If [G1,G2,G3,G5 all are AND gate && G4 = NOR && G1,G2 are connected && G2,G3 are connected && G3,G4 are connected && G4,G5 are connected] is the locality of a net then there is a higher possibility that it can be a Trojan.
Any	Trojan node's feature importance score is significantly higher than that of any other nodes.
Any	Trojan node mostly exists on a locale where it is surrounded by adjacent and connected AND gates.

most influential feature is $G2_AND$, followed by $G3_AND$. Connection between $G2$, $G3$ (Adj-23) is significant.

E. Explainability: Enhanced Findings

Fig. 3 shows the feature distribution analysis which highlights the model's decision-making process for Trojan and non-Trojan nodes (see Fig. 3A) as well as aids in improving its classification accuracy (see Fig. 3B & Table III). A clear distinction is observed, where Trojan nodes exhibit significantly higher feature values compared to non-Trojan nodes. This sharp difference between the feature scores allows the model to effectively differentiate Trojan nodes from non-Trojan nodes. Fig. 3B delves into finding false negatives within the classification from non-Trojan nodes. In this case, certain non-Trojan nodes show unexpectedly high feature values, almost similar to those of Trojan nodes. Finally, such instances are reclassified as true positives (non-Trojan into Trojan).

F. Generating Human-Readable Rules

The *Captum-Explainer* algorithm helps creating rules from model's decision process. In s35932_t200 benchmark, the gates $G1$, $G2$, $G3$, $G5$ all are AND gates (Fig. 2) which significantly influences the model's prediction toward identifying a Trojan node. This rule along with other benchmarks'

feature scores are translated into human-readable rules in Table IV. Additionally, the table highlights two common rules that are applicable across all benchmarks. We deduce that the presence of an AND gate in the local neighborhood plays a pivotal role in steering the prediction to detect a Trojan.

VI. CONCLUSION

Detecting malicious modifications in digital designs is crucial for ensuring trust and security of modern electronics driving the recent advances in Artificial Intelligence. State-of-the-art hardware Trojans detection solutions suffer from scalability concerns, lack detection precision/stability, and can be a victim of AI hallucinations. To mitigate these concerns, we propose SALTY that utilizes Jumping Knowledge mechanism to improve performance and an explainability-guided dynamic post-processing module for minimizing AI hallucinations. SALTY outperformed all SOTA Trojan detection frameworks across a large set of benchmarks. Future works will extend SALTY for detecting other threats (e.g. side-channel leakage, fault injection vulnerability).

VII. ACKNOWLEDGMENT

This material is supported by the National Science Foundation (NSF) under Grant No. 2350363 and Grant No. 2316399.

REFERENCES

- [1] P. Gaikwad, J. Cruz, P. Chakraborty, S. Bhunia, and T. Hoque, "Hardware ip assurance against trojan attacks with machine learning and post-processing," *ACM Journal on Emerging Technologies in Computing Systems*, vol. 19, no. 3, pp. 1–23, 2023.
- [2] T. Hoque, J. Cruz, P. Chakraborty, and S. Bhunia, "Hardware ip trust validation: Learn (the untrustworthy), and verify," in *2018 IEEE International Test Conference (ITC)*. IEEE, 2018, pp. 1–10.
- [3] H. Lashen, L. Alrahis, J. Knechtel, and O. Sinanoglu, "Trojansaint: Gate-level netlist sampling-based inductive learning for hardware trojan detection," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2023, pp. 1–5.
- [4] D. Cheng, C. Dong, W. He, Z. Chen, and Y. Xu, "Gnn4gate: A bi-directional graph neural network for gate-level hardware trojan detection," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022, pp. 1315–1320.
- [5] A. Imangholi, M. Hashemi, A. Momeni, S. Mohammadi, and T. E. Carlson, "Fast-go: Fast, accurate, and scalable hardware trojan detection using graph convolutional networks," in *2024 25th International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2024, pp. 1–8.
- [6] K. Hasegawa, K. Yamashita, S. Hidano, K. Fukushima, K. Hashimoto, and N. Togawa, "Node-wise hardware trojan detection based on graph learning," *IEEE Transactions on Computers*, 2023.
- [7] P. Chakraborty, J. Cruz, A. Alaql, and S. Bhunia, "Sail: Analyzing structural artifacts of logic locking using machine learning," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3828–3842, 2021.
- [8] S. Li, Y. Zhang, X. Chen, M. Ge, Z. Mao, and J. Yao, "A xgboost based hybrid detection scheme for gate-level hardware trojan," in *2020 IEEE 9th Joint International Information Technology and Artificial Intelligence Conference (ITAIC)*, vol. 9. IEEE, 2020, pp. 41–47.
- [9] L. Chen, C. Dong, Q. Wu, X. Liu, X. Guo, Z. Chen, H. Zhang, and Y. Yang, "Gnn4ht: A two-stage gnn based approach for hardware trojan multifunctional classification," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [10] K. Hasegawa, S. Hidano, K. Nozawa, S. Kiyomoto, and N. Togawa, "R-hdetector: Robust hardware-trojan detection based on adversarial training," *IEEE Transactions on Computers*, vol. 72, no. 2, pp. 333–345, 2022.
- [11] T. Mahfuz, S. Bhunia, and P. Chakraborty, "X-dfs: Explainable artificial intelligence guided design-for-security solution space exploration," *IEEE Transactions on Information Forensics and Security*, 2024.
- [12] M. T. Ribeiro, S. Singh, and C. Guestrin, "“ why should i trust you?” explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.
- [13] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins *et al.*, "Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai," *Information fusion*, vol. 58, pp. 82–115, 2020.
- [14] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.
- [15] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM computing surveys (CSUR)*, vol. 51, no. 5, pp. 1–42, 2018.
- [16] P. Chakraborty, J. Cruz, and S. Bhunia, "Sail: Machine learning guided structural analysis attack on hardware obfuscation," in *2018 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*. IEEE, 2018, pp. 56–61.
- [17] P. Chakraborty, J. Cruz, R. Almawzan, T. Mahfuz, and S. Bhunia, "Learning your lock: Exploiting structural vulnerabilities in logic locking," *IEEE Design & Test*, 2024.
- [18] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio *et al.*, "Graph attention networks," *stat*, vol. 1050, no. 20, pp. 10–48 550, 2017.
- [19] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *International conference on machine learning*. PMLR, 2018, pp. 5453–5462.
- [20] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *arXiv preprint arXiv:1903.02428*, 2019.
- [21] N. Kokhlikyan, V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan *et al.*, "Captum: A unified and generic model interpretability library for pytorch. arxiv," *arXiv preprint arXiv:2009.07896*, 2020.
- [22] S. Lundberg, "A unified approach to interpreting model predictions," *arXiv preprint arXiv:1705.07874*, 2017.
- [23] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *International conference on machine learning*. PMLR, 2017, pp. 3319–3328.
- [24] Trust-Hub, "Trust-Hub: A Community Resource for Hardware Security," accessed: 2024-11. [Online]. Available: <https://www.trust-hub.org>